

Proprietà	Valore
Canali	1, mono
Sample rate	16.000 Hz
Tipo campione	signed int16
Ordine byte	little-endian
Durata frame	40 ms
Campioni per frame	640
Byte per frame	1.280

Chiamata pubblica:

```
publish_audio_frame(
    session_id: str,
    sequence: int,
    captured_at_ns: int,
    pcm: bytes,
) -> bytes
```

Il record scritto nello stream audio:{session_id} contiene:

Campo Redis	Tipo/forma	Significato
sequence	intero codificato ASCII	ordine del frame
captured_at_ns	intero codificato ASCII	timestamp monotono
sample_rate	b"16000"	frequenza di campionamento
frame_ms	b"40"	durata del frame
pcm_s16le	1.280 byte	campioni audio grezzi

2. Redis → Worker

Il worker legge lo stream mediante consumer group. `parse_audio_frame()` rifiuta record incompleti, sample rate o durata inattesi e payload di dimensione errata. Il messaggio viene confermato con XACK soltanto dopo l'elaborazione riuscita.

`assemble_word_window()` riceve più record già nell'ordine di Redis, verifica che i numeri di sequenza siano contigui e concatena i rispettivi `pcm_s16le`. Il risultato è una registrazione completa della parola, non un singolo frame.

3. Worker → Cleaning

Interfaccia pubblica:

```
clean_audio(pcm_s16le: bytes) -> bytes
```

Precondizioni:

- finestra completa, mono, 16 kHz;
- campioni signed int16 little-endian;
- lunghezza pari e non vuota.

Postcondizioni:

- output mono, 16 kHz, signed int16 little-endian;
- regione vocale valida;
- durata compresa tra 0,35 e 15 secondi;
- picco massimo pari a -1 dBFS;
- output deterministico.

cleaning.py è l'adattatore del contratto. Converte temporaneamente int16 in float32 normalizzato e invoca audio_cleaning.py. La pipeline interna può filtrare, rilevare la voce, ridurre il rumore e normalizzare il livello. Prima di uscire dal boundary, l'audio viene nuovamente serializzato come int16.

4. Cleaning → Pronunciation engine

Interfaccia:

```
PronunciationEngine.evaluate(  
    pcm_s16le: bytes,  
    sample_rate: int,  
    expected_phonemes: list[str],  
    accepted_variants: list[list[str]],  
) -> PronunciationResult
```

Il primo argomento è esattamente l'output del cleaning. Il benchmark fonetico proviene da SQLite, mentre l'audio non viene salvato in SQLite.

5. SQLite

SQLite è il database persistente e contiene:

- benchmarks: parola, lingua, fonemi attesi, varianti e versione;
- attempts: sessione, benchmark, fonemi rilevati, confidenza, punteggio, versione del motore e timestamp.

Non contiene audio grezzo né audio pulito. Al termine dell'analisi, `save_attempt()` registra soltanto il risultato strutturato.

6. Risultato → interfaccia utente

Il worker pubblica l'evento JSON `pronunciation.evaluated` nello stream session: `{session_id}:events`. L'API inoltra gli eventi al client tramite:

```
WebSocket /sessions/{session_id}/events
```

7. Contratto verificato dai test

La suite costruisce gli stessi record che `publish_audio_frame()` scrive in Redis, inclusi metadati e frame da 1.280 byte. Verifica poi:

1. validazione e aggregazione di sequenze Redis contigue;
2. cleaning da PCM int16 a PCM int16;
3. formato, livello, durata, determinismo e rifiuto del silenzio;
4. passaggio dell'output al pronunciation engine;
5. caricamento del benchmark e salvataggio del risultato in SQLite.

In questo modo non esiste un formato audio speciale usato soltanto dai test.